# Islamic University of Technology

**Department of Computer Science and Engineering (CSE)**

B.Sc. in Software Engineering

**SWE 4304:** Software Project Lab I

Winter, 2024-2025

---

## Final Project Report

The 'Drim' Programming Language


**Group 7**

Md. Muntahi Hasan Akhiar, 230042118

S.M. Tahsinuzzaman Emon, 230042104

Tamim Mahrush Naser, 230042133

**Supervisor**

Farzana Tabassum,

Lecturer, CSE


Date: March 23, 2026

# Contents

**1 Problem Statement** **2**

**2 Potential Solution** **3**

**3 Implemented Features** **4**

**4 Class Diagram** **5**

**5 Tools and Technology** **6**

**6 Application Domain** **7**

**7 Project Timeline** **8**

**8 Contribution** **9**

**9 Future Work** **10**

**10 Source Repository** **11**

# 1 Problem Statement

Standard programming languages can be hard for beginners because they require manual memory management and strict variable declarations. This adds a lot of extra work when someone just wants to write and run code quickly, specially beginners.

Another problem is that most languages do not have built-in functions for things like physics formulas or unit conversions, forcing users to find and import external libraries.

Beyond these technical issues, we also wanted to address a personal motivation. The name **"Drim"** started as an inside joke and a meme in our student batch from our first day at university. We wanted to build something that would keep this memory alive through custom keywords like `drimIn` and `drimming`.

From an engineering perspective, we also wanted to master language design and memory management by building every component manually, without relying on third-party libraries.

# 2 Potential Solution

To solve these problems, a new interpreted language called "Drim" was created. It is designed to be simple and easy to use for beginners. **Drim** is a custom-built, lightweight language that runs as a Command Line Interface (CLI) tool. It uses a **Tree-Walk Interpreter**, which means it reads `.drim` files, understands the code, and runs it directly in real-time. Everything in Drim was built from scratch using **C++ 17** to ensure full control over the language pipeline.

In Drim, variables are created automatically when they are first used or assigned a value, and their types are handled by the language. The language includes built-in functions for physics calculations and unit conversions so that users don't have to worry about external libraries. To keep the code safe and reliable, the interpreter uses a system where function arguments are kept strictly inside their own local space. This ensures that even during recursion, variables do not interfere with each other.

# 3 Implemented Features

The project includes a complete system for reading, parsing, and running code. The flow of the language follows a standard interpreter pipeline, including a Lexer (Tokenizer), a Parser that builds an Abstract Syntax Tree (AST), and an Interpreter to execute the logic.

Figure 1: System Architecture Flow: From Source Code to Execution

The following features were implemented:

- **Input/Output:** Accepts user input through the `drimIn()` command and shows output using `wakeOut()`.
- **Automatic Variable Assignment:** If a value is assigned to a variable that hasn't been created yet, the system automatically creates it with the correct type.
- **Dynamic Type Casting:** Variables can hold different types like Strings, Integers, or Booleans, and the system handles the conversion automatically.
- **Mathematical Operations:** Full support for math like addition (+), subtraction (-), multiplication (*), division (/), modulo (%), and power ($\wedge$).
- **Built-in Physics Functions:** Direct support for formulas like mass-energy ($E = mc^2$), force ($F = ma$), and more.
- **Unit Conversion Engine:** Built-in tools to convert between things like inches and centimeters, Fahrenheit and Celsius, or kilometers and feet.
- **Control Flow:** Supports conditional logic (if-else) and loops (`drimming`).
- **Custom Syntax:** Uses unique keywords like `drimming` for loops and `wakeOut` for output to match the project's theme.
- **String Interpolation:** Allows variables to be put directly inside strings (like `"Value: {x}"`) for easier printing.
- **Error Reporting:** Provides clear error messages with exact locations when something goes wrong in the code.
- **Headless Execution:** Runs as a lightweight tool in the terminal without needing a complex interface.
- **Zero-Dependency Architecture:** Built from scratch in C++ 17 without using any external libraries for parsing.
- **Multi-Assignment:** Allows assigning values to multiple variables at once using commas.
- **Scoped Variables:** Ensures variables stay within the block where they were created.
- **User-Defined Functions:** Users can create their own functions that run during the program.

# 4 Class Diagram

The following class diagram represents the structure and relationships between the different components of the Drim language:

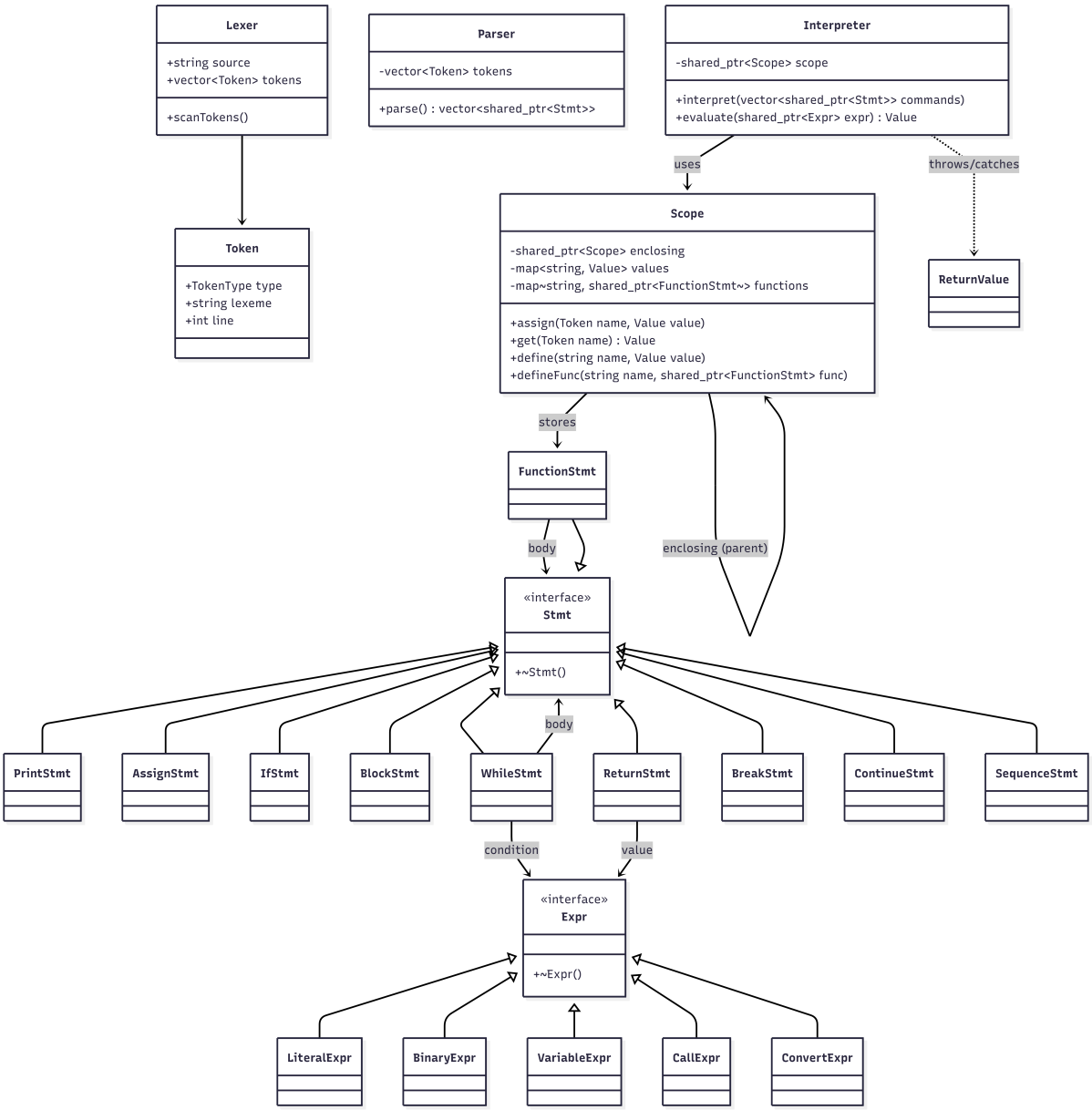


Figure 2: Project Class Diagram

# 5 Tools and Technology

The project used the following tools:

- **Programming Language:** C++ (Standard 17) for manual control and performance.
- **Build System:** CMake for managing the project and making it work on different systems.
- **Version Control:** Git and GitHub for working together and tracking changes.
- **IDE:** CLion and Visual Studio Code for writing and debugging the code.
- **Diagrams:** Mermaid.js and Gamma for creating charts and architecture maps.

# 6 Application Domain

The project belongs to the domain of **Custom Programming Language Design**. It is built to show how an interpreter works and to provide a specialized tool for people who need quick calculations for physics and general unit conversions. For those who want to start programming without worrying about complex syntax and memory management, Drim offers a simple and user-friendly environment.

# 7 Project Timeline

The project was built in stages, starting with simple text reading and moving to complex features like functions and loops.

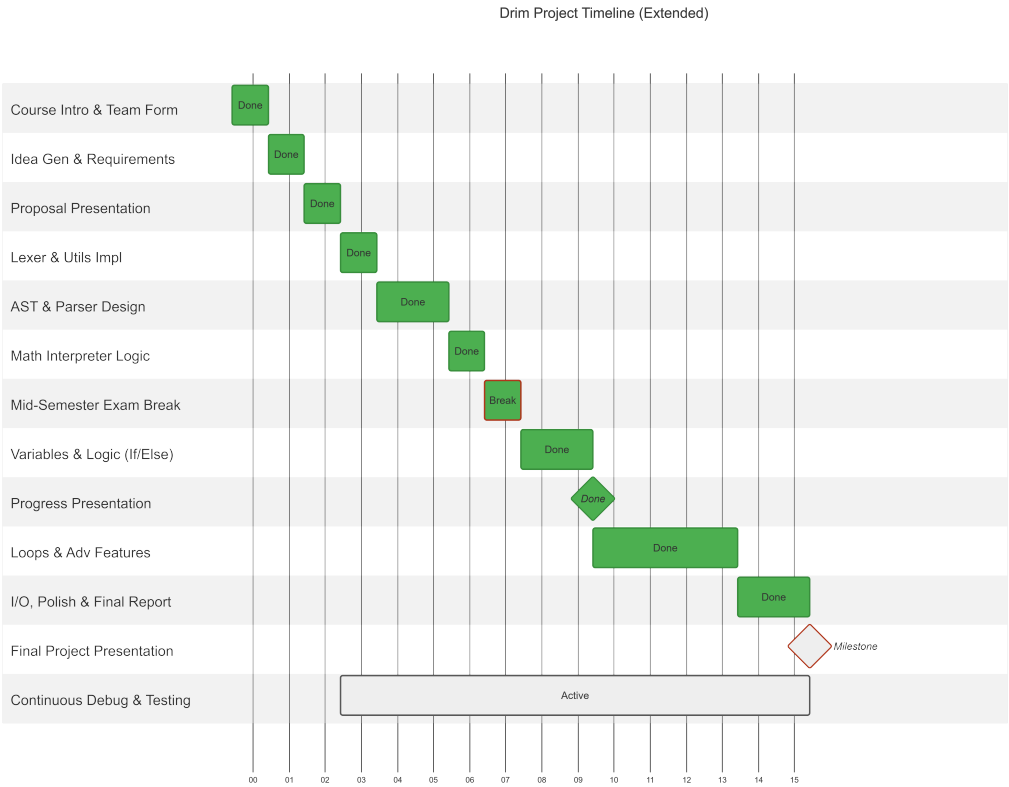


Figure 3: Project Development Timeline

# 8 Contribution

The work was divided among the team members as follows:

- **Md. Muntahi Hasan Akhiar:**
  - Created basic input and output systems.
  - Implemented variable management and automatic type casting.
  - Developed conditional logic and loops (`drimming`).
  - Added support for arrays and multi-assignment syntax.
  - Fixed modulo operations for math.
- **S.M. Tahsinuzzaman Emon:**
  - Implemented mathematical operations.
  - Built the physics function engine.
  - Added the Boolean data type.
  - Developed scoped variable logic.
  - Created the system for user-defined runtime functions.
- **Tamim Mahrush Naser:**
  - Added unit conversion functions.
  - Implemented data structure support.
  - Developed string interpolation and formatted output.
  - Worked on increasing internal calculation precision (int64).

# 9 Future Work

We plan to keep improving Drim by adding more features. This includes adding many more functions for physics and unit conversions to make the language more useful. We also want to refine the core system and add more complex data structures. Other students have already started contributing on their forks, such as adding color highlighting for keywords, which we might include in the future.

# 10 Source Repository

The code is available at:
`https://github.com/hasanakhiar/drim-lang`